

Technical Report: Book Metadata, Review & Insight API

1. Introduction

This project implements a data-driven REST API for managing book metadata, authors, user reviews, reading lists, and analytical insight endpoints. The application was designed to satisfy the coursework requirement for a complete web service that demonstrates structured API design, persistent data storage, authentication, testing, deployment, and professional documentation.

The system supports two main user groups. General users can browse books, submit and manage reviews, maintain reading lists, and receive personalised recommendations. Administrators can create, update, and delete authors and books, as well as recalculate derived rating fields. The final implementation is available both locally and via a live Render deployment, which makes the project suitable for demonstration during the oral presentation.

This report explains the project data sources, data model, technical architecture, API design, testing strategy, deployment approach, version control practice, and use of Generative AI.

2. Data Sources and Data Modelling

The project uses a relational data model implemented with SQLAlchemy [2]. SQLite is used for local development and test isolation, while PostgreSQL is used for deployed hosting. The core entities are **User**, **Author**, **Book**, **Review**, and **ReadingListEntry**.

The **User** entity stores identity and access-control fields such as username, email, password hash, activity status, admin role, and creation timestamp. A user can create many reviews and many reading list entries. The **Author** entity stores author-level metadata including name, country, and biography. A single author can be associated with many books. The **Book** entity stores bibliographic and analytical fields such as title, ISBN, publication date, description, cover image URL, main genre, average rating, ratings count, and a nullable foreign key to an author. The **Review** entity records the rating and review text provided by a user for a specific book, with a database constraint limiting ratings to the range 1 to 5. The **ReadingListEntry** entity links a user to a book that they wish to save, and a uniqueness constraint prevents duplicate entries for the same user-book pair.

The application uses a real public metadata source rather than a purely hand-written dataset. The bundled dataset `data/raw/open_library_books.csv`

is a curated 20-book export generated from the Open Library Search API [4] by `scripts/fetch_open_library_dataset.py`. The import utility `scripts/import_books_from_csv.py` maps this CSV into the project schema and creates authors automatically, while also preventing duplicate book insertion through ISBN checks. A second utility, `scripts/seed_demo_users_and_reviews.py`, optionally creates example users and a small amount of review activity on top of the imported public books so that authentication and analytics features can be demonstrated more easily.

Open Library is used as an offline public-data source rather than a live dependency during request handling [4]. This choice reduces runtime complexity and external failure risk while still allowing the project to demonstrate the use of genuine public data. The imported records remain traceable to an identifiable external source, which strengthens the academic credibility of the dataset and supports the coursework requirement to reference data origins.

3. Architecture and Technology Choices

FastAPI was selected as the core web framework because it provides asynchronous request handling, automatic OpenAPI documentation, strong type-hint integration, and clear support for dependency injection [1]. This makes it well suited to a coursework project that needs a clean, testable service layer and a professional documentation story through Swagger UI and ReDoc.

SQL was chosen over a NoSQL alternative because the project domain is strongly relational. Books, authors, users, reviews, and reading lists depend on stable foreign-key relationships, and the analytics endpoints rely on joins and aggregate queries. SQLAlchemy 2.x is used as the ORM and session-management layer [2], while Alembic is used for migration tracking [3] so that schema changes can be applied consistently across local and deployed environments.

The codebase is organised into clear layers. Route handlers are defined under `app/api`, data models under `app/models`, request and response schemas under `app/schemas`, and shared services such as configuration, security, and exception helpers under `app/core`. This structure keeps endpoint logic compact and makes the application easier to test, maintain, and explain in the report and presentation.

Authentication uses JSON Web Tokens created after a successful login request. Passwords are hashed rather than stored directly, and role-based checks restrict admin-only operations such as managing authors and books. The use of dependency-based authentication in FastAPI keeps security checks reusable and consistent across endpoints.

4. API Design and Implementation

The API follows a resource-oriented structure under the `/api/v1` prefix. Core resources are exposed through predictable REST-style routes such as `/authors`, `/books`, `/reviews`, `/reading-list`, and `/analytics`. CRUD operations use conventional HTTP methods: `GET` for retrieval, `POST` for creation, `PUT` for updates, and `DELETE` for removal. A dedicated health endpoint at `/api/v1/health/ping` supports deployment verification.

The authentication flow includes registration, login, and a current-user endpoint. The login endpoint follows the OAuth2 password flow expected by Swagger UI, which improves the usability of the interactive documentation. Once authenticated, a user can manage their own reviews and reading-list items. Administrative operations are guarded through explicit role checks.

The books resource supports pagination, text search, filtering by author or genre, and ordering by title, average rating, or rating count. Reviews automatically trigger recalculation of derived book statistics so that the stored `average_rating` and `ratings_count` remain consistent with the underlying review data. This makes analytics queries cheaper and simplifies list views.

In addition to standard CRUD operations, the project includes analytical endpoints intended to demonstrate higher-value API functionality. These include rating distribution by book, top-rated genres, trending authors, and recommendation generation for the currently authenticated user. The recommendation logic is intentionally explainable rather than opaque: it uses the genres and ratings from a user's prior activity to suggest books that match demonstrated preferences.

Error handling was standardised so that API consumers receive a unified response shape of the form `{"error": {"code": "...", "message": "...", "details": {...}}}`. This improves client-side predictability, makes the behaviour easier to document, and ensures validation and permission errors are communicated consistently.

5. Testing, Deployment, and Version Control

Automated testing is implemented with `pytest`, `pytest-asyncio`, and `httpx`. The test suite runs against an isolated in-memory SQLite database so that tests are repeatable and do not depend on local development data. Current coverage includes authentication behaviour, CRUD flows for authors and books, review creation and update behaviour, reading-list permissions, consistent error formatting, and analytics endpoints such as rating distribution and recommendations. This level of testing is especially important because the project uses

asynchronous database access and role-based security checks, both of which are areas where regressions can be subtle.

The application is deployed on Render [5], with PostgreSQL used for the hosted database. The repository includes a `render.yaml` blueprint and environment-driven configuration through `.env` and deployment variables. The production service is available at <https://book-insight-api.onrender.com>, and the live interactive documentation is exposed at <https://book-insight-api.onrender.com/api/v1/docs>. Running Alembic migrations as part of the build process ensures that the database schema is in sync with the application code on deployment.

Version control was managed through Git, with the project stored in a public GitHub repository [6] at <https://github.com/John2968/book-insight-api>. Using Git throughout development supported incremental implementation, testing, refactoring, documentation updates, and deployment preparation. The repository also provides transparent evidence of development progress and supports the coursework expectation that version-control practice should be part of the presentation.

For submission purposes, the main project links are as follows:

- **Public GitHub repository:** <https://github.com/John2968/book-insight-api>
- **API documentation in the repository (Markdown source):** <https://github.com/John2968/book-insight-api/blob/main/docs/api-documentation.md>
- **API documentation in the repository (PDF after export):** <https://github.com/John2968/book-insight-api/blob/main/docs/api-documentation.pdf>
- **Presentation slides:** [https://leeds365-my.sharepoint.com/:p:/r/personal/sc23jl3_leeds_ac_uk/Document]
- **Live API documentation:** <https://book-insight-api.onrender.com/api/v1/docs>

6. Use of Generative AI

Generative AI was used in line with the module’s Green Light policy. The formal declaration is included in **Appendix A** of this report, and selected exported AI conversation logs are included in **Appendix B**.

AI assistance was used to explore alternative API structures, compare architectural options, draft implementation scaffolding, improve error-handling consistency, generate and refine automated tests, and improve the organisation of the written documentation. AI support was also useful when troubleshooting dependency issues and evaluating how best to present the data-ingestion approach using a real public dataset.

However, the final design decisions, implementation choices, corrections, verification steps, and all submission judgments remained the responsibility of the author. AI-generated suggestions were treated as draft material rather than authoritative answers and were checked against framework documentation, runtime behaviour, and coursework requirements before being kept.

One important challenge during the project was distinguishing between suggestions that were convenient and suggestions that were genuinely suitable for the coursework requirements and the actual runtime behaviour of the application. This was particularly relevant when dealing with dependency issues, migration configuration, testing isolation, and the use of real public data. A key lesson learned was that Generative AI is most valuable when used as a structured assistant for exploration, debugging, and reflection, but its outputs still need to be validated carefully through testing, documentation checks, and independent technical judgment.

7. Limitations and Future Work

The current deployment is intentionally lightweight. It does not include caching, background task processing, rate limiting, API-key access, or CI/CD automation. The recommendation logic is heuristic and explainable rather than machine-learning based. In addition, Open Library is used as a curated offline import source rather than as a live enrichment dependency during requests, so some metadata fields remain incomplete when the public source does not provide them directly.

There are several realistic directions for future improvement. The system could enrich records with additional live metadata sources, support cover and description backfilling, extend the analytics layer with more comparative and time-series views, and introduce stronger deployment automation. Recommendation quality could also be improved through collaborative filtering, embedding-based similarity, or hybrid ranking methods. These extensions were not necessary for the coursework deliverable but would be sensible next steps for a production-oriented version of the system.

References

1. FastAPI. *FastAPI Documentation*. Available at: <https://fastapi.tiangolo.com/>
2. SQLAlchemy. *SQLAlchemy Documentation*. Available at: <https://docs.sqlalchemy.org/>

3. Alembic. *Alembic Documentation*. Available at: <https://alembic.sqlalchemy.org/>
4. Open Library. *Search API Documentation*. Available at: <https://openlibrary.org/dev/docs/api/search>
5. Render. *Render Documentation*. Available at: <https://render.com/docs>
6. GitHub repository for this project. Available at: <https://github.com/John2968/book-insight-api>

Appendix A: GenAI Declaration

I declare that I used Generative AI tools in this assessment in accordance with the module's Green Light policy. The tools were used to support design exploration, implementation drafting, debugging, testing, and documentation improvement. All final technical decisions, code selection, verification steps, and submitted wording remained my own responsibility.

Tools Used

Tool / product	Purpose
Cursor (AI-assisted editor)	Exploring endpoint structure, drafting FastAPI routes and schemas, reviewing database and authentication patterns, generating and refining tests, improving documentation structure, and helping draft the technical report and declaration.
ChatGPT (used occasionally)	Clarifying framework behaviour, reviewing wording, and discussing implementation alternatives before selecting the final approach.

How GenAI Was Used

- **Design and architecture:** AI was used to compare API structures, discuss endpoint naming, explore error-response standardisation, and reason about how to combine CRUD functionality with analytics and recommendation endpoints.
- **Implementation support:** AI provided draft code patterns for FastAPI dependency injection, JWT-based authentication, SQLAlchemy models, CRUD handlers, CSV ingestion utilities, and exception helpers. These suggestions were adapted, corrected, and integrated selectively.
- **Testing and debugging:** AI was used to draft test ideas and help diagnose issues such as dependency installation problems, async fixture setup, database session isolation, and validation behaviour.
- **Documentation:** AI supported the drafting and restructuring of the README, API documentation, technical report, and declaration so that the project could be presented more clearly and professionally.

My Own Contribution

- I selected the project topic, defined the final scope, and chose the technology stack.
- I decided the final data model, security model, endpoint set, and deployment strategy.
- I reviewed all AI-generated suggestions before using them and rejected or changed outputs that did not match the coursework requirements or actual runtime behaviour.
- I ran the application, verified the API behaviour, executed the tests, and corrected implementation issues where necessary.
- I take responsibility for the correctness of the submitted code and documents.

Verification and Limitations

- AI suggestions were treated as draft support rather than as authoritative answers.
- I checked AI-assisted outputs against framework documentation, runtime results, and the coursework brief.
- Where AI suggestions were inaccurate, incomplete, or poorly matched to the project, I revised them before keeping them.
- AI did not replace my own understanding of the system; it was used to accelerate iteration and improve quality.

Appendix B: Selected AI Conversation Logs

The following examples summarise selected conversations that show how I used Generative AI throughout the project. They are presented in a simple question-and-answer form to reflect the actual style of interaction used during planning, implementation, debugging, deployment, and submission preparation.

B1. Choosing the Project Direction

Question: How can this project be shaped so that it aims for a high mark in both technical quality and GenAI usage, rather than being only a minimal CRUD API?

Answer: This project can be framed as a complete Book Metadata and Review Analytics API rather than a simple CRUD exercise. It should be planned in stages, including architecture, data modelling, CRUD resources, analytics, testing, deployment, and documentation. This direction supports a broader and more ambitious scope, including higher-value features such as analytics and recommendations.

B2. Deciding the Technical Stack

Question: Which technology stack would be most suitable if the API needed to be modern, well documented, easy to demonstrate, and suitable for deployment?

Answer: A suitable stack is FastAPI, SQLAlchemy, Alembic, JWT authentication, SQLite for local development, and PostgreSQL for deployment. This combination supports automatic API documentation, modular code structure, testing, and hosted deployment. It is also consistent with the final implementation described in the main report.

B3. Planning the Data Model and Endpoint Structure

Question: How should the project be broken down into entities and endpoints so that the API feels complete rather than fragmented?

Answer: The project can be structured around a relational design based on `User`, `Author`, `Book`, `Review`, and `ReadingListEntry`, together with REST endpoints for authentication, books, authors, reviews, reading lists, and analytics. It is also useful to include pagination, filtering, sorting, rating distributions, and personalised recommendations. This structure aligns with the final API design and helps the service go beyond the minimum endpoint requirement.

B4. Finding and Integrating a Real Public Dataset

Question: What would be a better dataset strategy than relying only on hand-written sample data?

Answer: A better strategy is to use a genuine public metadata source and build a workflow in which data can be fetched, normalised, stored in CSV form, and imported into the application database. This supports the decision to use Open Library as the public metadata source and to implement the workflow around `scripts/fetch_open_library_dataset.py`, `data/raw/open_library_books.csv`, and `scripts/import_books_from_csv.py`.

B5. Rethinking the Seeding Strategy

Question: After adopting a real public dataset, how can demonstration data still be used without implying that every part of the database is fully real?

Answer: Real imported book metadata can be separated from optional demonstration user behaviour. Instead of inventing the whole dataset, it is better to keep the books real and add only a small number of example users and reviews for testing and presentation. This matches the final use of `scripts/seed_demo_users_and_reviews.py`, which adds demo activity on top of imported public books.

B6. Debugging Dependencies, Migrations, and Runtime Errors

Question: How should the runtime and installation errors encountered during development be interpreted and fixed?

Answer: These errors can be investigated by tracing each failure back to its likely cause and then testing targeted fixes. In this project, that included missing packages such as `aiosqlite` and `email-validator`, Alembic migration problems, requirements-file encoding issues, and password-hashing compatibility warnings. Working through the tracebacks in this way supported the final stable setup process described elsewhere in the report.

B7. Improving Tests and Error Handling

Question: How can the API be made stronger in terms of testing and error handling, rather than only implementing the happy path?

Answer: The API can be strengthened by writing tests for authentication, CRUD flows, analytics, permission checks, validation behaviour, and error formatting. It is also useful to return a consistent error-response shape so that code, tests, and documentation align with each other. This is consistent with the final pytest suite and the unified error format used across the API.

B8. Deployment and Final Submission Preparation

Question: Once the API was working, how should the final stage of the project be handled in terms of deployment, documentation, and submission preparation?

Answer: The final stage can be handled by working through Render deployment, environment variables, the difference between local SQLite and deployed PostgreSQL, how Swagger UI and ReDoc can be used for demonstration, and how the technical report should be restructured so that the GenAI declaration and selected conversation evidence appear in the appendices. This supports the final deployment and the submission-ready report structure described in this document.